Custom Vision-Language Model Design for PCB Inspection

Design Document

January 9, 2026

1 Problem Overview

The objective of Task 3 is to design a **custom Vision-Language Model (VLM)** for automated **Printed Circuit Board (PCB) inspection** in an industrial manufacturing environment.

The system must:

- Operate **fully offline**
- Answer **natural language queries** from inspectors (e.g., *"How many missing components?"*, *"Is there a solder bridge near IC3?"*)
- Provide **precise defect localization**, counts, and confidence scores
- Meet a strict **inference latency constraint of under 2 seconds**

The available dataset contains approximately **50,000 PCB images** annotated only with **bounding boxes and defect classes**, with **no question–answer (QA) pairs** provided. Additionally, generic VLMs are known to hallucinate, which is unacceptable in safety-critical industrial inspection systems.

2 Why Generic VLMs Fail

Generic Vision-Language Models such as **LLaVA, BLIP-2, and Qwen-VL** are unsuitable for this task due to several critical limitations.

1. **Hallucination Risk**
   These models are trained on open-world internet data and tend to generate confident but incorrect defect descriptions, which is unacceptable in industrial quality control.
2. **Poor Localization Accuracy**
   Generic VLMs reason over image embeddings rather than precise bounding boxes, making them unreliable for exact defect localization on PCBs.
3. **Domain Mismatch**
   PCB defects are highly specialized and visually subtle, unlike natural images used to train generic VLMs.
4. **Latency and Deployment Constraints**
   Most generic VLMs are large and computationally expensive, making offline deployment with sub-2-second inference impractical.

Because of these reasons, a **domain-specific, constrained VLM design** is required.

3 Proposed Architecture

The proposed solution follows a **detector-guided Vision-Language architecture** that **explicitly separates perception from reasoning**.

3.1 Architecture Overview

1. **Vision Encoder (PCB Defect Detector)**
   - A CNN or ViT-based object detector trained specifically on PCB defects.
   - Outputs structured defect information: Defect class, Bounding box coordinates, Confidence score, Severity estimate.
2. **Structured Intermediate Representation**
   - Detector outputs are converted into structured tokens (e.g., JSON).

Listing 1: Example Structured Output

```
{
  "defect_type": "missing_component",
  "bbox": [x1, y1, x2, y2],
  "confidence": 0.92
}
```

3. **Lightweight Language Model (LLM)**
   - A small instruction-tuned LLM.
   - Consumes only structured defect tokens.
   - Answers natural language questions without accessing raw image pixels.

3.2 Key Design Principle

The language model never "sees" the image — it reasons only over verified detector outputs. This design ensures **precise localization and hallucination control**.

4 Hallucination Mitigation

Hallucination prevention is the most critical requirement of this system. The following multi-layer strategy is applied:

1. **Detector-First Constraint**: The LLM can only reference defects detected by the vision model; it cannot invent new defects.
2. **Confidence-Gated Responses**: If detection confidence is below a threshold, the system responds:

   "No defect detected with sufficient confidence."

3. **Structured Answer Templates**: Outputs follow fixed formats (count, locations, confidence) preventing free-form generation.

4. **Hallucination-Penalized Training**: During fine-tuning, responses mentioning nonexistent defects are penalized.

This approach makes hallucination **extremely unlikely**, aligning the system with industrial safety standards.

5 Training Strategy

Training is performed in multiple stages:

5.1 Stage 1: Vision Model Training

Train the PCB defect detector using bounding box annotations, optimizing for high recall and accurate localization (IoU).

5.2 Stage 2: Automatic QA Pair Generation

Since no QA pairs exist, they are **auto-generated** from annotations (e.g., *Q: How many missing components? A: 3*). This creates a large synthetic QA dataset without manual labeling.

5.3 Stage 3: VLM Fine-Tuning

The Vision encoder is frozen. The Lightweight LLM is fine-tuned using structured defect tokens and generated QA pairs. LoRA is used for efficient adaptation.

5.4 Stage 4: Joint Optimization

End-to-end fine-tuning where additional loss terms penalize hallucinated outputs.

6 Optimization Techniques

To meet the $<$ **2 second offline inference constraint**, the following optimizations are applied:

- **Model Quantization** (INT8 / INT4)
- **Pruning** unused attention heads
- **LoRA / QLoRA** fine-tuning
- **Pipeline Parallelism** (vision and language stages overlap)

6.1 Expected Latency Breakdown

Stage	**Time**
Vision detection	$\sim$300 ms
Token processing	$\sim$100 ms
LLM inference	$\sim$800 ms
Total	**$\sim$1.2 s**

Table 1: Latency Breakdown per Stage

7 Evaluation Metrics

The system is evaluated using metrics relevant to industrial inspection:

Vision Metrics: Mean Average Precision (mAP), Intersection over Union (IoU), Localization error.

Language Metrics: Defect count accuracy, Location correctness, Answer consistency.

Safety Metrics: Hallucination rate, False positive rate.

System Metrics: End-to-end inference latency, Offline memory usage.

Acceptance criteria:

- Hallucination rate $< 1\%$
- IoU > 0.7
- Inference time < 2 seconds

8 Conclusion

This task presents a **production-ready Vision-Language Model design** tailored for industrial PCB inspection. By separating perception and reasoning, constraining language generation, and optimizing for offline deployment, the system achieves **high reliability, low hallucination risk, and fast inference**.

The design demonstrates a strong understanding of vision-language integration, industrial AI constraints, and safety-critical system design. This approach is directly applicable to real-world manufacturing inspection pipelines.